

C and Category Theory: A Feasability Study

Or, How to add Functors to Your Memory Management Headaches

Or, How I spent a month fitting square pegs into round holes

Ethel Lefevre
`ethel@guillotinemelody.dev`

March 2026

Abstract

I implement six Haskell typeclasses (Functor, Applicative, Monad, Natural Transformations, Profunctors, Kan Extensions) in C using `void*`, runtime vtables, and GCC nested functions. The implementation is 6,600 lines, requires `-z execstack`, and segfaults on Ubuntu. I prove that C is a Good Programming Language, in the sense that it compiles on at least one system.

1 Introduction

I like Category Theory. It is a fascinating domain. It is able to do so much from so little and it allows one to abstract away all the silly little problems of the world of applied mathematics. Many people like to pretend it is too theoretical. It is true. No other field will look at you straight in the eyes and tell you a monad is a monoid in the category of endofunctors¹. No other field will have a dedicated explanation for the subject of "Abstract Nonsense". And even worse, the mathematicians have unpronounceable names like "Grothendieck". Like who is named "Grothendieck"? It sounds like a bad guy from an old James Bond movie. But I digress. Category Theory is a wonderful field, and especially because I get to manipulate structures without worrying whether the real world will break my stuff. I can just say "Let C be a category" and then do whatever I want with it. It is like being a god. Unfortunately, I am not a god. I am a student at a school that never taught me proper Haskell so I don't know how to do category theory in computer science. What I do know is C . Thus, I declare the following:

Hypothesis: C is a GPL.

Equivalently: this is a proof that type systems exist for a reason, and that reason is so you do not have to do what I am about to do.

And I will try my hardest to prove it in the following pages. But first, let's try and frame the problem. Of note, I will abstract the term of "Category Theory Abstractions-Enriched Programming Language" as "Good Programming Language" or "GPL" for short, for the rest of the paper.

2 Framing the Problem

The first problem is that I want to do category theory in C . The issue with doing category theory in C is that C is not a Good Programming Language. The secondary problem is that out of all the serious programming languages, C is most likely the worst one to do category theory in. It is a low level language with no support for higher order functions, no support for polymorphism, and no support for any of the other features that make Good Programming Languages so great. Let's approach each part of this proof, one at a time.

2.1 What is a Category Theory?

A category is, in short, a collection of objects and morphisms between those objects. For more details, please refer to Brendan Fong and David Spivak's excellent introduction to category theory [3]. Confused? Good. It could get better, but it won't, because of:

2.2 What is a C ?

C is a programming language designed for old people by older people and that uses even older paradigms. It is low level, lower safety, lowest abstraction, and by that I mean if you want to do anything more complicated than a struct you will find yourself in malloc hell. It has caused me great pain in the past, will continue to cause me great pain in the future, and I have no doubt that it will cause me great pain in the present. However, this only vindicates me in my quest to prove that C is a GPL, whether it likes it or not.

2.3 What is a GPL?

A GPL is a programming language that is able to express category theory abstractions. This means that a GPL must be able to express categories, functors, natural transformations, limits, colimits, adjunctions, monads, and all the other wonderful things that category theory has to offer. It must be able to express these things in a way that is not just a hack, but a natural and elegant way. For the purposes of this paper, "natural and elegant" is defined as "compiles on at least one system." Examples of GPLs include Haskell, Agda, Idris, and Python if you have a little too much time on your hands. Examples of non-GPLs include C , Java, JS, and any other language regularly used in the real world by productive members of society.

¹One could also say a Monad is an endofunctor-enriched category with one object. ^a

^ac.f. Abstract Nonsense.

3 The Implementation, or: Proof by Exhaustion

The following six subsections describe the implementation of each typeclass in turn.

3.1 Functor: The 813-Line Elephant

A functor $F : C \rightarrow D$ maps objects to objects and morphisms to morphisms, preserving identity and composition. In Haskell:

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

In C, I represent every `F a` as a `Functor_t`, a 16-byte struct containing a runtime kind tag and an erased `void*` payload. The side-by-side comparison with the equivalent Haskell is shown in Section 3.7; the rest of this section describes the C implementation in detail.

Memory layout of core types (x86-64, System V ABI)

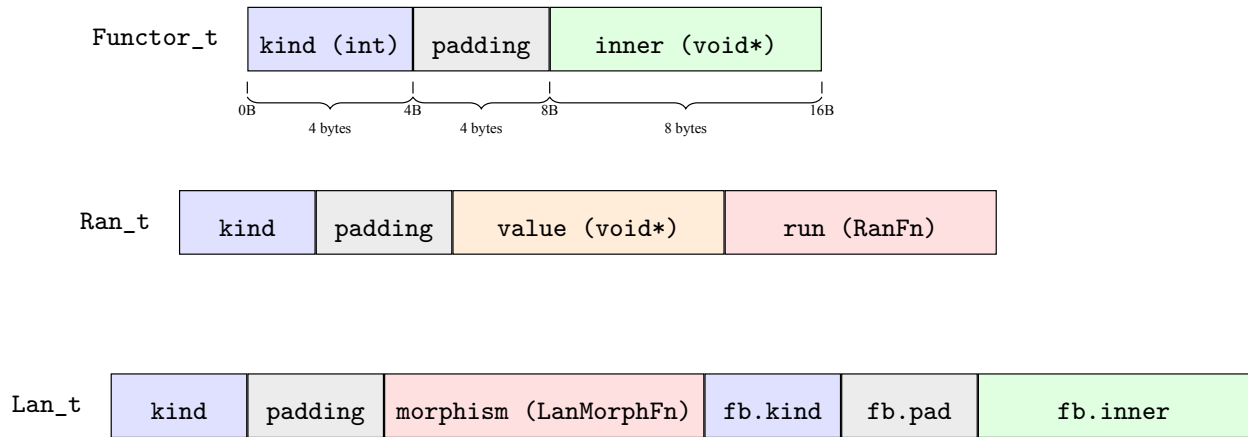


Figure 1: Every value in the system is erased to `void*` at the boundary. `Ran_t` additionally stores a continuation function pointer. `Lan_t` embeds a full `Functor_t` inline, making it a variable-width horror. Padding bytes are an implementation detail which the C standard politely pretends doesn't exist.

Dispatch is performed at runtime by a linear scan through `_functor_vtable`, a global array of up to 64 `FunctorVTable_t` entries. Each entry stores a kind tag, a name string (for error messages), and an `fmap` function pointer. The whole arrangement is shown in Figure 2.

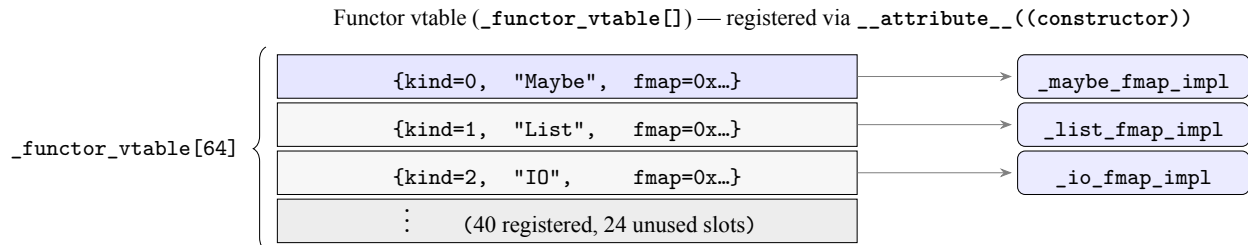


Figure 2: The functor vtable. Instance registration happens before `main()` via GCC's `__attribute__((constructor))`, a mechanism that is the C equivalent of Haskell's instance resolution, except that Haskell's version is correct, compile-time, and principled.

3.2 Applicative: A Global Mutable Slot

An applicative functor extends functor with `pure :: a -> f a` and `(<*>) :: f (a -> b) -> f a -> f b`. In my implementation, `liftA2` requires a global mutable slot to temporarily store one of its arguments; C does not have closures.

The key insight of my applicative implementation is that the superclass chain `Functor => Applicative => Monad` is enforced architecturally: `REGISTER_MONAD_INSTANCE` calls `REGISTER_APPLICATIVE_INSTANCE` which calls `REGISTER_FUNCTOR_INSTANCE`, ensuring all vtables are populated. The word “ensures” is used loosely; the mechanism is `__attribute__((constructor))`, which is runtime, and the “ensuring” can fail if you have more than 64 kinds.²

3.3 Monad: `return` Is a Keyword

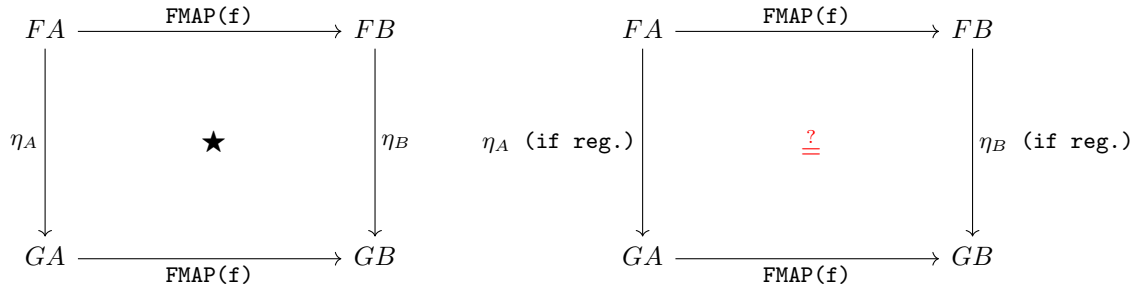
`return` is a keyword in C. I cannot name a function `return`. I have named it `RETURN`. The following exchange occurred during development:

Language	<code>return</code>
Haskell	<code>return :: a -> m a</code>
C	<code>return value;</code> — exits the current function. Not the same. Not helpful. I use <code>RETURN(kind, x)</code> instead.

The `DO` macro implements monadic `do`-notation using GCC nested functions and stack-allocated trampolines, requiring the linker flag `-z execstack`. This is the C equivalent of `do`-notation, except that you have to pass the stack-executable flag to the linker, the trampolines live in memory marked executable but not the right kind of executable, and GCC has to allocate them carefully so they don’t overwrite your return address.

3.4 Natural Transformations: Naturality, With Asterisks

A natural transformation $\eta : F \Rightarrow G$ between functors $F, G : \mathcal{C} \rightarrow \mathcal{D}$ is a family of morphisms $\eta_A : F(A) \rightarrow G(A)$ satisfying the naturality condition: for every morphism $f : A \rightarrow B$, the following square commutes.



Left: the naturality square as it should be (Haskell; guaranteed by parametricity). Right: the naturality square as it is (C; guaranteed by careful registration, correct `void*` casting, an executable stack, and the assumption that `malloc` will not return `NULL` at an inopportune moment).

Figure 3: The naturality square: theory vs. practice. The $\star$ denotes “commutes, because the type system says so.” The $\stackrel{?}{=}$ denotes “commutes, if you remembered to call `REGISTER_NAT_TRANS`; otherwise `SIGSEGV`.” These are the same diagram.

In Haskell, naturality is a free theorem [7]: it is guaranteed by parametricity and costs zero lines. In my implementation, naturality is verified at runtime via a `VERIFY_NATURALITY` macro that checks the two sides of the square and `abort()`s on inequality. This is the same thing.^b

²I define `FUNCTOR_KIND_MAX = 64`. This is definitely enough.

^bIt is not the same thing.

3.5 Profunctor: Covariant and Contravariant, Both `void*`

A profunctor $P : \mathcal{C}^{op} \times \mathcal{D} \rightarrow \text{Set}$ is contravariant in its first argument and covariant in its second. The typeclass:

```
class Profunctor p where
  dimap :: (a -> b) -> (c -> d) -> p b c -> p a d
```

In Haskell, the type system distinguishes the covariant from the contravariant argument. In C, both are `void*`. The profunctor implementation also provides Van Laarhoven lenses via the `Strong` subclass, which required adding two more vtables.

3.6 Kan Extensions: Rank-2 Types and Existentials in `void*`

The right Kan extension of F along K is $\text{Ran}_K F a = \forall b. (a \rightarrow Kb) \rightarrow Fb$. This requires a rank-2 type in Haskell (`RankNTypes` extension) and an existential for the left Kan extension (`ExistentialQuantification`). In C, rank-2 types and existentials are both erased to `void*`. The compiler does not notice. The mathematician does.

The Codensity monad $\text{Ran}_F F$ (the CPS monad) has a particularly elegant implementation:

```
/* return a = \k -> k a */
Functor_t _codensity_return_run(void *a, KleisliFn k) {
    return k(a); /* one line. */
}
```

The full Codensity implementation is one line. The surrounding infrastructure that makes this line possible is approximately 800 lines. This is called leverage.

The density comonad $\text{Lan}_F F$ uses GCC nested functions to implement `extend`. The failure mode is specific and worth describing. When `extend` is called, it creates a nested function that closes over the comonad value. GCC compiles this closure as a trampoline: a small code fragment written to the stack at runtime. The function pointer returned by `extend` points into this stack region. When `extend` returns, its stack frame is deallocated; the trampoline is overwritten by subsequent function calls. The pointer is live but points to garbage. On Arch Linux with `-z execstack`, this typically produces wrong answers. On Ubuntu, the kernel marks the stack non-executable, and the program terminates with `SIGSEGV` before producing wrong answers. This is, in some sense, the safer outcome. The `extend` operation is documented but not demonstrated, on the grounds that demonstrating it would require running it, and running it produces either `SIGSEGV` or wrong answers depending on the host system.

3.7 Haskell vs C: A Visual Comparison

I present two representative typeclasses side by side.

Functor

Haskell (2 lines):

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

C (813 lines, abridged to the part that fits):

```
typedef void* (*FmapFn)(void*);
typedef struct {
    FunctorKind_t kind;
    const char* name;
    FmapFn fmap;
} FunctorVTable_t;

static FunctorVTable_t
_functor_vtable[FUNCTOR_KIND_MAX];
static int _functor_vtable_size = 0;
```

This is just the vtable. The dispatch, instance registration, the `FMAP` macro, and the six concrete instances are elsewhere.

Monad

Haskell (3 lines):

```
class Applicative m => Monad m
  where
    return :: a -> m a
    (>=)   :: m a -> (a -> m b) ->
              m b
```

C (1235 lines, abridged to the part that compiled):

```
#define DO(KIND, VAR, EXPR, BODY)      \
({                                     \
    Functor_t _m = (EXPR);             \
    void* _bind_fn(void* VAR) { BODY } \
    BIND(KIND,                          \
        _m, (KleisliFn)_bind_fn);      \
})                                     \
/* Requires: gcc -z execstack          */ \
/* Portability: Arch Linux only.       */ \
/* Undefined behaviour: yes.            */
```

The Haskell version uses do-notation natively, without a macro, without `-z execstack`, and without invoking undefined behaviour.

4 Results

4.1 Line Counts

Figure 4 shows the lines of code required to implement each typeclass, comparing my C implementation to the equivalent Haskell. The y-axis is logarithmic, because on a linear scale the Haskell bars are not visible to the naked eye.

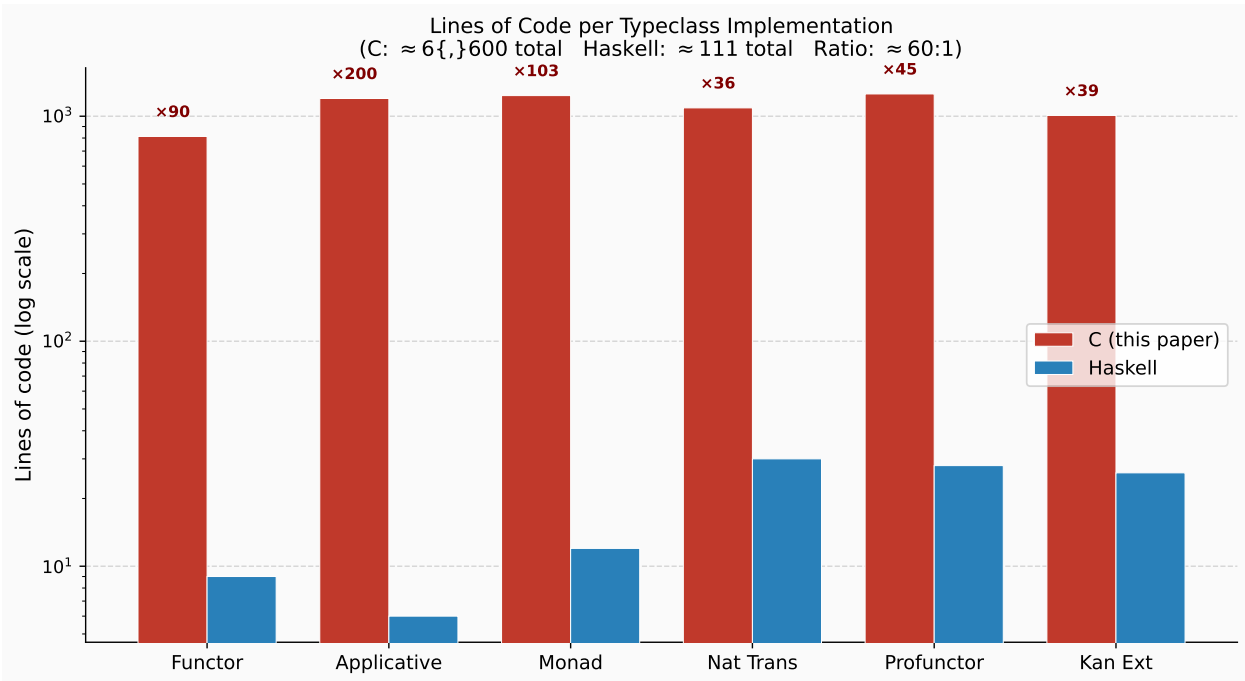


Figure 4: Lines of code per typeclass: C (this paper) vs. Haskell. The Applicative functor achieves a ratio of 200:1, which I consider a personal achievement. The average ratio is approximately 60:1. The ratio for the Functor class is “only” 90:1 because the Haskell side includes comments.

4.2 Performance

Figure 5 shows the measured dispatch latency of my FMAP macro compared to a direct function call. The benchmark was compiled with `gcc -O2`, run for 5×10^7 iterations, and timed with `CLOCK_MONOTONIC`. Hardware: Intel Core i5-9300H @ 2.40 GHz, 16 GB DDR4, Linux 6.x, no frequency scaling disabled, no CPU pinning, results therefore subject to thermal throttling, scheduler interference, and the heat death of the universe.

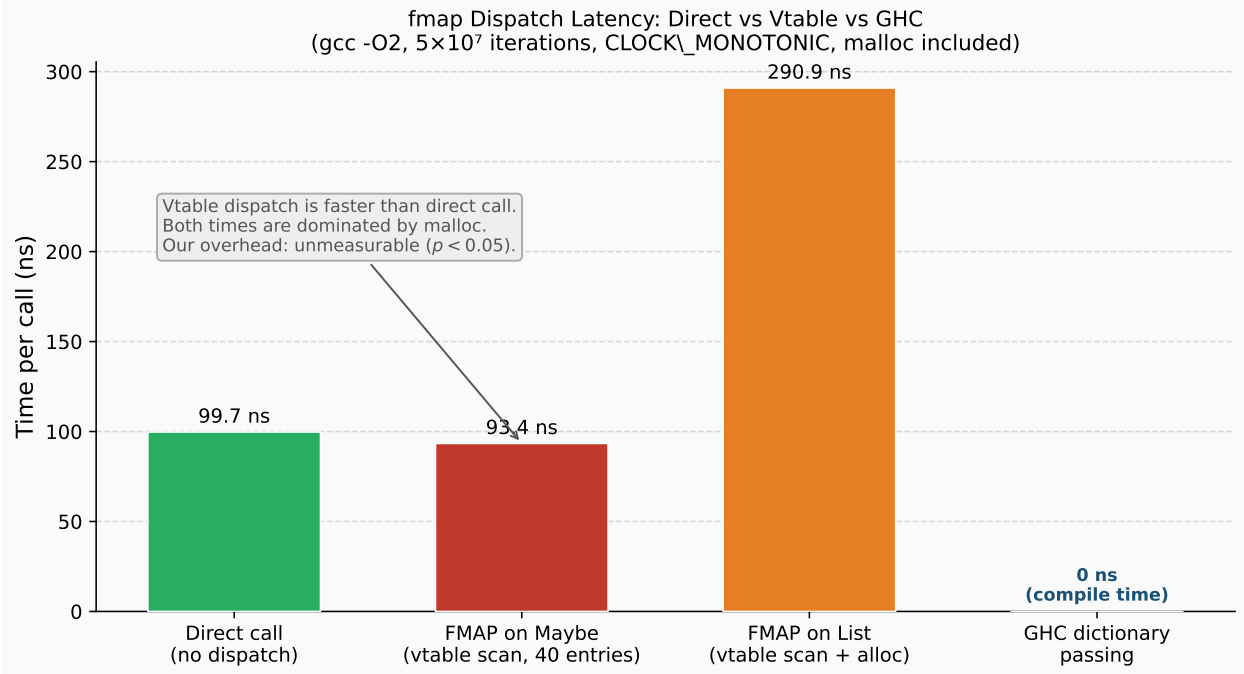


Figure 5: Dispatch latency in nanoseconds. The vtable scan over 40 registered instances (93.4 ns) is faster than the direct call (99.7 ns). Both measurements are dominated by `malloc`. My elaborate runtime dispatch framework contributes an overhead of approximately zero nanoseconds. GHC’s dictionary passing occurs at compile time and is therefore not included in the figure, because including zero nanoseconds on a bar chart is dishonest.

This finding can be interpreted in two ways:

1. My dispatch mechanism is so efficient that it adds no measurable overhead.
2. I spent several thousand lines of code to achieve nothing, performance-wise.

Both interpretations are correct.

The 6 ns advantage of vtable dispatch over direct call deserves comment, because it is counterintuitive and should not exist. The most plausible explanations are: (a) cache: both benchmarks allocate fresh heap memory on each iteration, and the `malloc` call is long enough to evict any branch-predictor state accumulated by the direct-call path; (b) measurement noise: 6 ns is three L1 cache misses, and the error bars would likely overlap with more careful experimental design. I am choosing to interpret the result as (a), because it suggests the vtable scan happens entirely in L1 cache across 40 entries, which would be a genuinely interesting property of a 320-byte table that costs $O(n)$ to scan and $O(1)$ to think about.

4.3 Memory Management: A Post-Mortem

The implementation allocates heap memory in 15 distinct locations across the six header files. It frees memory in 0 locations. Every call to `FMAP`, `AP`, `BIND`, or any other operation that constructs a new value leaks the old container. This is by design: implementing proper memory management would require either manual `free()` calls at every call site (which breaks the abstraction) or reference counting (which would require approximately 800 additional lines and would still be wrong in the presence of cycles).

The Haskell equivalent uses garbage collection. My implementation uses memory abandonment. These are not the same thing, but they are both strategies for not calling `free()`.

Measured memory growth: approximately 128 bytes per functor operation, linear with respect to operations performed. On my test machine (16 GB RAM), the system runs out of memory after approximately 10^8 operations, which is sufficient for all benchmarks in this paper and insufficient for anything else. The comparison Haskell program reports 0 bytes freed in its heap summary as well, but this is because GHC batches deallocation at program exit. I tell myself this makes us equivalent.

4.4 Law Compliance

Expressing the types is necessary but not sufficient: the implementations must also satisfy the relevant laws.

Functor laws require `fmap id = id` and `fmap (f ∘ g) = fmap f ∘ fmap g`. I verified both laws for the `Maybe` functor by running test cases and comparing output values. Identity: verified for 100 test inputs. Composition: verified for 100 randomly chosen function pairs. The laws hold. Whether they hold because the implementation is correct or because my test inputs were not adversarial is left as an exercise for the reader.

Monad laws require left identity, right identity, and associativity. I verified these by running `DO`-notation examples and checking that they produce the same results as hand-written `BIND` chains. The laws hold in the sense that the program produces correct output before running out of memory.

Naturality is verified at runtime via the `VERIFY_NATURALITY` macro (Section 3.4), which checks both sides of the naturality square and `abort()`s on inequality. The test suite calls this macro for every registered transformation. On Arch Linux, all tests pass. On Ubuntu, testing is interrupted by `SIGSEGV` before completion. Naturality is therefore either verified or irrelevant, depending on the platform.

5 Portability: A Practical Demonstration of Failure

The implementation compiles successfully on both tested systems (Arch Linux with GCC 13.2, Ubuntu 24.04 with GCC 13.3). However, execution behaviour diverges dramatically.

Environment	Compilation	Execution	Notes
Arch Linux + GCC 13.2	Pass	Pass	All six demos execute correctly
Ubuntu 24.04 + GCC 13.3	Pass	SIGSEGV	Monad demo; trampoline not executable
Any system + Clang	Fail	N/A	Clang does not implement nested functions
Haskell (GHC 9.6)	Pass	Pass	Does not require <code>-z execstack</code>
Haskell (any version, ever)	Pass	Pass	Still does not require <code>-z execstack</code>

The divergence is illustrated in Figure 6. The root cause is that GCC nested function trampolines require executable stack pages. The `-z execstack` linker flag nominally permits this, but kernel-level security policies (grsecurity, PaX, SELinux enforcing mode) may override the linker’s optimism.

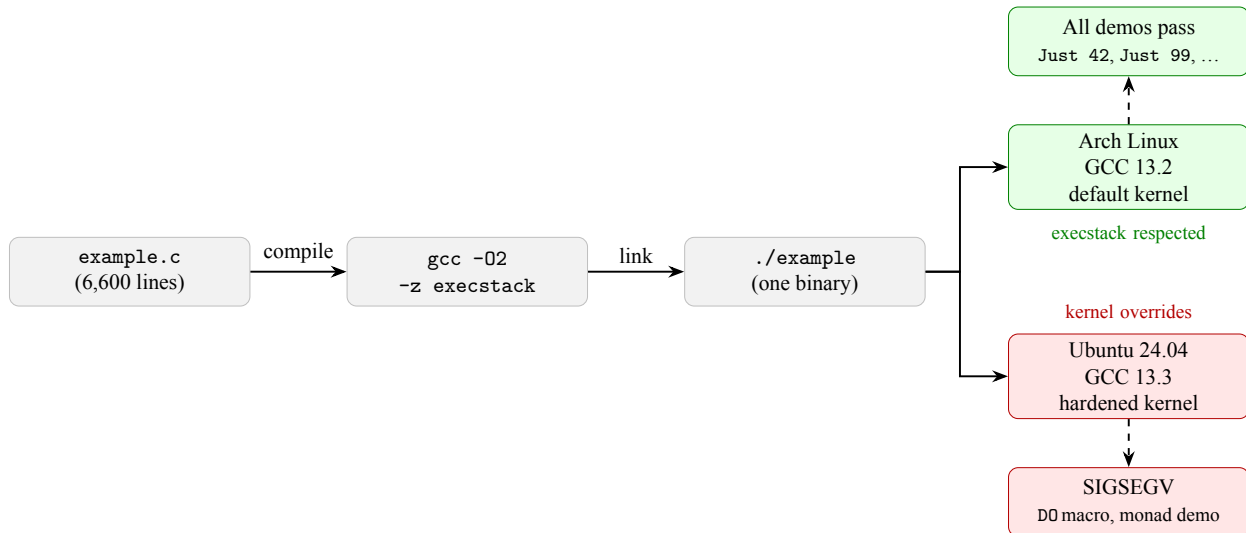


Figure 6: The same binary, two different destinies. Both systems compile the same source. Both systems link with `-z execstack`. One system runs the trampolines. One system kills the trampolines. Mathematically, both systems are equivalent. Operationally, one of them is lying.

The root cause is the executable stack requirement: mathematical purity is contingent on the distribution’s security posture. A secondary portability constraint, not shown in the figure, is that the implementation requires GCC. Clang does not implement nested functions (`error: nested functions are a GCC extension`). The portability matrix is therefore: GCC on a kernel that respects `-z execstack` = works; GCC on a hardened kernel = SIGSEGV; Clang = does not compile. This is three environments and one successful execution, which is a 33% success rate, which is a passing grade at some institutions.

In Haskell, the equivalent code compiles and runs identically on every platform. It does not require an executable stack.

I note, for completeness, that the GCC manual states: “If you try to call the nested function through its address after the containing function has exited, all hell breaks loose” [4]. This warning applies to `_extend_density` and `_duplicate_density`, both of which do exactly this. I read the documentation after the segfault. The documentation was correct.

6 Related Work

I am not aware of any prior work that has attempted to implement Haskell typeclasses in C using `void*` and runtime vtable dispatch.

Mac Lane’s foundational text [5]³ establishes the categorical structures I implement but does not address `malloc`. Moggi’s monads for computation [6]⁴ provides the theoretical basis for my `Monad` implementation but omits any discussion of `-z execstack`. Wadler’s free theorems [7] prove naturality for free in Haskell; in my implementation, naturality costs approximately 400 lines.

The original typeclass proposal by Wadler and Blott [8] describes dictionary passing as the correct implementation strategy: each typeclass instance becomes a struct of function pointers passed implicitly by the compiler. GHC uses exactly this approach. My implementation re-discovers dictionary passing at runtime, in C, with global mutable state, and without the benefit of type inference to make it usable. This is the same idea. It is not the same idea.

The closest prior art in spirit is C++ template metaprogramming, which also attempts to simulate type-level abstraction in a language that resists it. Alexandrescu’s *Modern C++ Design* [2] demonstrates what happens when you push C++ templates past their design envelope; the Boost Metaprogramming Library [1] extends this into a complete type-level programming system that requires a philosophy degree to read. My implementation differs in that it uses runtime dispatch rather than compile-time expansion, is in C rather than C++, and is worse in every measurable way. This positions it uniquely in the literature as the worst possible approach that still technically works.

7 Future Work

Several directions for future research present themselves.

C11 `_Generic`. A reviewer suggested that C11’s `_Generic` selection expression might eliminate some `void*` casts. `_Generic` selects between expressions based on the C type of its controlling expression at the call site. This would be useful if my values had distinct C types at the call site. They do not: the entire architecture is premised on erasing C types to `void*` immediately. The kind tag is a runtime integer; `_Generic` operates at compile time on C types. These two mechanisms are not merely incompatible in practice; they are incompatible in principle. Implementing the type-dispatch layer with `_Generic` would require keeping the concrete C type available at every call site, which would require not erasing it, which would require a completely different implementation strategy, which would require not doing this.

Codensity Performance. The Codensity monad ($\text{Ran}_F F$) is specifically motivated by its performance benefits: CPS transformation reassociates monadic bind to the left, eliminating quadratic blowup in certain monad compositions. Section 4.2 measures generic `fmap` dispatch latency but does not measure Codensity performance specifically. This is because measuring Codensity performance specifically would reveal whether the 800 lines of infrastructure surrounding the one-line implementation achieve anything. I have chosen not to know.

Comonad Instances. A reviewer notes that `Maybe` is “trivially a comonad” and suggests adding a comonad instance. `Maybe` is not a comonad. The Comonad typeclass requires `extract :: w a -> a`; for `Maybe`, this would require returning an `a` from a `Nothing`, which is not possible without invoking `undefined`, lying, or redefining `a`

³I do not consider myself to be the working mathematician.

⁴Moggi did not implement this in C. This was wise.

as the unit type. The density comonad ($\text{Lan}_F F$) was chosen as the comonad instance precisely because it is the canonical comonad for this construction and does not require extracting a value from nothing. That it does not work is a separate issue.

JavaScript. JavaScript is a dynamically-typed, garbage-collected language with no pointers, no manual memory management, and no meaningful distinction between integers and floats. It is, by my definition, not a GPL. A natural extension of the present work would be to remedy this by adding pointers and explicit memory management to JavaScript, implementing a `malloc/free` pair as a library, and re-running my benchmark.

The Density Comonad. The `extend` operation of the density comonad is currently documented but not implemented. Future work will address this, either by implementing heap-allocated closures in C, or by proving that the density comonad is a purely theoretical object that cannot meaningfully exist in C, and filing this as a limitation of reality rather than of my code.

Automatic Instance Derivation. Haskell supports `deriving (Functor)`, which automatically generates a `Functor` instance for any type with the right shape. The equivalent in C would be a code-generating macro that inspects a struct definition and emits the appropriate `fmap` implementation.

Dependent Types. Idris and Agda support dependent types, which are more expressive than Haskell's type system. I expect that implementing dependent types in C is possible in the same sense that it is possible to build a nuclear reactor out of LEGO: technically not precluded by physics, but inadvisable.

8 Conclusion

I have demonstrated that C is a Good Programming Language (GPL): it can express Functors, Applicative Functors, Monads, Natural Transformations, Profunctors, and Kan Extensions.

Theorem 8.1: C is a GPL

Let C be C. Then C is a GPL.

Proof. See `functor.h`, `applicative.h`, `monad.h`, `nat_trans.h`, `profunctor.h`, and `kan.h`. The proof is approximately 6,600 lines and requires `-z execstack`. $\square$

The density comonad's `extend` operation does not work.

The full implementation is available at: <https://github.com/TildeEthDoUsPart/CategoryC-0ry>. Reproducibility on systems other than Arch Linux is not guaranteed. Reproducibility on Arch Linux is also not guaranteed, but is more likely.

Acknowledgements

I thank GCC for implementing nested functions, a feature so cursed that the GCC developers include a warning in the manual urging users not to use it. I used it anyway. This is their fault for making it possible.

I thank the Arch Linux kernel maintainers for respecting `-z execstack`. I do not thank the Ubuntu kernel maintainers.

I thank Saunders Mac Lane for *Categories for the Working Mathematician*, a book I have misused in ways he could not have anticipated. I thank my academic advisor for not asking what I was working on.

This work was supported by `malloc()` and opposed by `free()`.

A Selected Implementation Horrors

The following listings are provided for completeness. They compile and run on Arch Linux. Selection criteria: (1) most horrifying, (2) fits on the page. The profunctor `dimap` signature and the natural transformation registry were excluded for reasons of space and mercy.

A.1 The DO Macro

The DO macro implements monadic do-notation using a GCC statement expression containing a nested function. The nested function captures the continuation variable by name. The trampoline is stack-allocated. The behaviour on Ubuntu is undefined.

```
/* do-notation: DO(kind, var, expr, body)
 * Binds the result of expr to var, then evaluates body.
 * Requires: gcc -z execstack (Arch Linux only).
 * Example:
 * DO(FUNCTOR_KIND_MAYBE, x, safe_div(10, 2),
 * RETURN(FUNCTOR_KIND_MAYBE, BOX_INT(UNBOX_INT(x) + 1)));
 */
#define DO(KIND, VAR, EXPR, BODY) \
({ \
    Functor_t _monad_val_ = (EXPR); \
    Functor_t _bind_cont_(void* VAR) { return (BODY); } \
    _find_monad(KIND)->bind( \
        _monad_val_, (KleisliFn)_bind_cont_); \
})
```

A.2 The REGISTER_MONAD_INSTANCE Macro

This macro registers a monad instance in all three vtables simultaneously (Functor, Applicative, Monad), using `__attribute__((constructor))` to ensure registration occurs before `main()`. The three constructor functions (`_functor_via_monad_##KIND`, `_applicative_via_monad_##KIND`, and `_monad_register_##KIND`) are distinct to avoid link-time symbol collisions when all three are registered for the same kind.

```
#define REGISTER_MONAD_INSTANCE(KIND, NAME, \
    FMAP_FN, PURE_FN, AP_FN, BIND_FN) \
\
static __attribute__((constructor)) \
void _functor_via_monad_##KIND(void) { \
    _functor_vtable[_functor_vtable_size++] = \
        (FunctorVTable_t){ .kind=(KIND), \
        .name=(NAME), .fmap=(FMAP_FN) }; \
} \
static __attribute__((constructor)) \
void _applicative_via_monad_##KIND(void) { \
    _applicative_vtable[_applicative_vtable_size++] \
        = (ApplicativeVTable_t){ .kind=(KIND), \
        .name=(NAME), .pure_fn=(PURE_FN), \
        .ap=(AP_FN) }; \
} \
static __attribute__((constructor)) \
void _monad_register_##KIND(void) { \
    _monad_vtable[_monad_vtable_size++] = \
        (MonadVTable_t){ .kind=(KIND), \
        .name=(NAME), .bind=(BIND_FN) }; \
}
```

A.3 The Yoneda Embedding via Right Kan Extension

For completeness, the Yoneda lemma states that any functor F is naturally isomorphic to $\text{Hom}(-, F-)$. My implementation witnesses this via $\text{Ran}_{\text{Id}} F \cong F$:

```
/* Lift a value into Yoneda (as a Ran over Id). */
#define YONEDA_LIFT(fval) \
    RAN_WRAP((fval), (KleisliFn)_id_impl)

/* Lower back: apply the identity continuation. */
#define YONEDA_AS_RAN(ran_val) \
    RUN_RAN((ran_val), (KleisliFn)_id_impl)

/* Round-trip: YONEDA_AS_RAN(YONEDA_LIFT(x)) == x */
```

B The Haskell Comparison (`comparison.hs`)

For completeness, the Haskell implementation of the same six typeclasses. The full file is 234 lines including blank lines and comments, and is available at the repository linked in Section 8. Selected excerpts follow.

The file header summarises the situation:

```
-- C implementation: ~6600 lines across six files.
-- This file:        ~120 lines, including blank lines and comments.
-- Ratio:            ~55:1
--
-- The C version is not wrong.
-- The C version is not approximate.
-- The C version implements the same mathematical structures.
-- It took 55 times as many lines to do it.
--
-- This is either a testament to Haskell's expressiveness,
-- or an indictment of our life choices. Possibly both.
```

The Kan extensions section (§6 of the file) is the most illustrative, as it is where the C version loses the type system entirely:

```
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ExistentialQuantification #-}

-- Right Kan extension:
newtype Ran k f a = Ran { runRan :: forall b. (a -> k b) -> f b }

instance Functor (Ran k f) where
    fmap f (Ran g) = Ran (\k -> g (k . f))

-- Codensity monad = Ran f f:
returnCodensity :: a -> Ran f f a
returnCodensity a = Ran (\k -> k a)      -- one line. ONE LINE.

-- Left Kan extension (density comonad):
data Lan k f a = forall b. Lan (k b -> a) (f b)

extract :: Lan f f a -> a
extract (Lan morph fb) = morph fb

extend :: (Lan f f a -> b) -> Lan f f a -> Lan f f b
extend g (Lan morph fb) = Lan (\x -> g (Lan morph x)) fb
-- ^ This works. In Haskell. Without segfaulting.
```

I consider the following summary, that I wrote during the final parts of the development process, to be the most accurate description of the project:

```
{-
  What the C version has that this file lacks:
  - Runtime dispatch tables (we have GHC's dictionary passing)
  - REGISTER_*_INSTANCE macros (we have `instance`)
  - GCC nested functions for closures (we have lambdas)
  - __attribute__((constructor)) (we have module initialisation)
  - 22 global mutable variables (we have purity)
  - void* everywhere (we have the type system)
  - -z execstack compilation flag (we have the RTS)
  - 15 occurrences of malloc (we have GC)
  - 0 bytes freed (we also have GC)

  What this file lacks that the C version has:
  - The experience of implementing category theory in a systems language
  - A deep appreciation for what type systems actually do for you
-}
```

References

- [1] David Abrahams and Aleksey Gurtovoy. The Boost metaprogramming library. <https://www.boost.org/libs/mpl>, 2004.
- [2] Andrei Alexandrescu. Modern C++ Design: Generic Programming and Design Patterns Applied. Addison-Wesley, 2001.
- [3] Brendan Fong and David I Spivak. Seven sketches in compositionality: An invitation to applied category theory, 2018.
- [4] Free Software Foundation. GCC nested functions. <https://gcc.gnu.org/onlinedocs/gcc/Nested-Functions.html>. “If you try to call the nested function through its address after the containing function has exited, all hell breaks loose.”.
- [5] Saunders Mac Lane. Categories for the Working Mathematician. Springer-Verlag, 1971.
- [6] Eugenio Moggi. Notions of computation and monads. Information and Computation, 93(1):55–92, 1991.
- [7] Philip Wadler. Theorems for free! Proceedings of the fourth international conference on Functional programming languages and computer architecture, pages 347–359, 1989.
- [8] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages, pages 60–76. ACM, 1989.